Agile Methodology

Data Science Bootcamp

NASHVILLE SOFTWARE SCHOOL

Lesson Objectives

1. Know the key features of Agile Methodology

Qualities of Effective Teams

- Clear, straightforward communication
- Shared load
- Adaptability to changing circumstances
- Reliability
- Supportiveness
- Shared goal
- Collaborativity and cooperation

Agile Methodology

Agile is a popular method of project management that originated in software development and is now very common in most tech career fields. Characteristics of this approach include:

- Managing a team project by breaking it up into several phases or sprints
- Clearly defining the goal for each phase/sprint
- Providing structure for team communication and work assignments
- Ongoing opportunity to reflect, share feedback, and improve the process

There is no shortage of resources describing the Agile process, but here's a place to start if you want to read more:
https://www.agilealliance.org/agile101/

Defining Elements of the Agile Workflow

- Sprints
- Minimally Viable Product (frequently referred to as the MVP)
- Retrospectives (frequently referred to as Retros)
- Daily Standups
- Kanban Boards

Sprints

- Defined period of time to complete a chunk of a larger project
- Starts with a set of goals and ends with a retrospective
- Opportunity to test out new approaches to team workflow / interaction defined in previous retrospectives
- Each individual should have a specific product to work on during each sprint as a piece of the greater project

Minimum Viable Product

- The basic components needed to deliver the required insights / functionality are defined as the MVP
- Extra goals above and beyond the MVP that would be nice, but are not necessary can be identified as “stretch goals”
- Goals, MVP, workflow, etc. can be updated periodically based on accomplishments, improved understanding, and/or as external factors shift

Daily Standup

- A **short** (10-15 minute) conversation where everyone discusses their progress and can troubleshoot issues as a group. Each person should touch on:
  - What they have done since the last standup
  - What they plan to do next
  - What impediments are they facing
- **Should not include code walkthroughs or demos**, just high level description of progress
- In depth conversation and detailed code help should be held until after the standup
- Not a substitute for regular conversation about work / issues that arise. It is to make sure no issue goes unaddressed for more than a class

Aim to do a standup at the beginning of every class when working on a group project during class

Retrospective

- A structured way to look back at the previous sprint, evaluate team performance, and outline any adjustments to improve performance in the next sprint
- Should be collaborative
- The main points to discuss and record during a retrospective are:
  - What should **the team** keep doing?
  - What should **the team** start doing?
  - What should **the team** stop doing?
  - Specific action steps for the following sprint with a person assigned to each